PROJECT REPORT | 專案報告

# AI Coding 生命週期管理導入專案報告

AI Coding Lifecycle Management & Vibe Coding Governance

Howard Liao Ph.D.

| | |
|---|---|
| **文件編號** | AIC-PRJ-01 |
| **版次** | V1.0 |
| **發行日期** | 2026-10-21 |
| **文件性質** | 專案報告（導入規劃與試行計畫） |
| **適用範圍** | 本公司軟體開發、測試、維運及資訊安全相關單位 |
| **主責單位** | 資訊單位（AI Coding 導入專案小組） |

| 制定 | 審核 | 核准 |
|---|---|---|
| | | |
| 日期： | 日期： | 日期： |

本報告為專案規劃文件，文中之時程、指標門檻與審核層級均為建議範例，實施前應依組織實際規模、系統風險與內部核決權限調整後核定。

# 修訂履歷與目錄

| 版次 | 日期 | 修訂內容 | 修訂者 |
|---|---|---|---|
| V1.0 | 2026-10-21 | 初版發行：專案目標、AI Coding Lifecycle、審核程序、SCM 規範、試行計畫 | 專案小組 |

## 目錄


1 專案摘要 3

2 專案背景與問題 4

3 專案目標、範圍與成功標準 5

4 Vibe Coding 使用邊界 6

5 AI Coding Lifecycle 管理架構 7

6 Pattern & Framework 與 AI 程式品質審查 8

7 審核程序與風險分級 Review Matrix 9

8 SCM 規範與稽核證據 10

9 落地三步驟 11

10 專案組織與權責 12

11 專案時程與里程碑 13

12 成效衡量指標 14

13 專案風險與因應 15

14 專案交付成果 16

15 結論與下一步 17

附 訪談與檢核題綱、參考資料 18


Howard Liao Ph.D.

# 1　專案摘要

> 本專案在 10 週內建立 AI Coding 的治理流程，讓 AI 加快開發速度的同時，維持程式品質、資訊安全與可稽核性。核心原則是：AI 可以產出與初審，合併與上線必須由具名人員核准並負責。

DORA 2025 報告指出，AI 是既有工程能力的放大器：流程健全的團隊獲益，流程薄弱的團隊則會更快累積問題。因此本專案的重點不在挑選工具，而在建立從需求、開發、測試、審查到部署的治理流程。

## 核心結論

| # | 結論 | 說明 |
|---|---|---|
| 1 | Vibe Coding 可以用，但要劃定邊界 | 適用於原型、概念驗證與內部小工具；正式系統必須回到受控的 AI Coding Lifecycle。 |
| 2 | AI 只能建議，不能核准 | AI 可產生程式、測試與初步審查意見；合併主幹與上線須由具名人員核准。 |
| 3 | Pattern 決定 AI 審查的品質上限 | Pattern 與 Framework 不是硬性前提，但文件化後，AI 才能審查分層、交易、權限與 API 契約等高價值問題。 |
| 4 | 能用工具強制的，不交給 AI | 格式、依賴方向、已知弱點與 Secret 交給 Linter、架構測試與掃描工具；AI 專注語意與設計判斷。 |
| 5 | 從小範圍試行開始 | 選一個低風險系統試行兩個 Sprint，以數據決定是否擴大導入。 |

Howard Liao Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。

# 2　專案背景與問題

AI Coding 工具已普遍進入開發流程，產出速度明顯提升，但審查、測試與發布流程若未同步調整，品質與資安成本會被轉嫁到後段。專案啟動前觀察到的主要問題如下：

| 問題 | 現象 | 影響 |
| --- | --- | --- |
| 使用方式不一致 | 各人自行選用 AI 工具與使用方式，缺乏共同規範與標註 | 無法追蹤 AI 參與範圍，問題難以歸因 |
| Vibe Coding 流入正式系統 | 以「能跑」為標準的原型程式，未經重構與測試即進入正式環境 | 維護困難，潛藏資安與資料風險 |
| 審查負荷上升 | AI 讓變更量與 PR 大小增加，人工 Review 成為瓶頸 | 審查流於形式，缺陷漏網 |
| 規範只存在於人腦 | 架構 Pattern 與框架慣例未文件化，AI 無從對照 | AI 審查意見偏淺、偏雜，甚至與架構衝突 |
| 資料與 Secret 邊界不清 | 程式碼、設定或資料可能被貼入未經核准的 AI 工具 | 機敏資訊外洩風險 |
| 缺乏衡量基準 | 沒有導入前後的效能與品質數據 | 無法判斷 AI 是否真正帶來效益 |

Howard Liao PhD.

本文件為內部管制文件，未經核准不得複製或對外提供。

# 3　專案目標、範圍與成功標準

## 專案目標

1. 建立 AI Coding Lifecycle，明確定義每個階段 AI 與人的責任。
2. 建立依風險分級的審核程序（Review Matrix）與 SCM 規範。
3. 將架構 Pattern 與 Coding Guideline 文件化，轉成 AI 可讀的規則檔。
4. 建置自動檢查與安全閘門，讓 AI 審查聚焦於語意判斷。
5. 完成兩個 Sprint 的試行，以數據作為擴大導入的決策依據。

## 專案範圍

| 納入範圍 | 不納入範圍 |
| --- | --- |
| AI Coding 使用政策、Vibe Coding 邊界 | AI 工具採購議價與合約談判 |
| Lifecycle、Review Matrix、SCM 規範 | 既有系統的全面重構 |
| 規則檔、PR 範本、自動檢查與安全閘門設定 | 非軟體開發部門的 AI 使用（如文書、客服） |
| 一個低風險系統的兩個 Sprint 試行 | 全公司所有系統同時導入 |

## 成功標準

| 項目 | 標準（建議） |
| --- | --- |
| 流程落實 | 試行期間所有合併至主幹的變更，均經 PR、自動檢查與具名人工核准 |
| AI 標註 | 試行系統的 PR 均填寫 AI 使用欄位 |
| 品質不退步 | 變更失敗率與程式返工率不高於導入前基準 |
| 資安 | Secret 外洩與高風險弱點漏網為零 |
| 效率 | 交付前置時間縮短，且 Review 時間未因變更量增加而拉長 |

Howard Liao Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。

# 4　Vibe Coding 使用邊界

Vibe Coding 指以自然語言描述需求，讓 AI 產生大部分程式碼，開發者主要看結果是否「能跑」，而不逐行理解程式內容。此概念由 Andrej Karpathy 於 2025 年初提出。它的價值在速度，風險在於沒有人真正理解、驗證並為程式負責。

| 面向 | Vibe Coding | 受控 AI Coding |
|---|---|---|
| **開發者角色** | 描述需求、接受結果 | 設計、驗證、對最終程式碼負責 |
| **程式理解** | 低，以執行結果判斷 | 高，逐段理解並審查 |
| **測試** | 多為手動試跑 | 自動化測試，涵蓋邊界與異常情境 |
| **審查** | 通常沒有 | AI 初審加人工 Code Review |
| **版控與稽核** | 鬆散，常為大批次變更 | 小批次 Commit、PR、核准紀錄 |
| **適用範圍** | 原型、POC、個人或內部小工具 | 正式產品、核心系統、對外服務 |

## 使用邊界

| 等級 | 適用情境 |
|---|---|
| 可以使用 | 概念驗證、UI 原型、一次性資料處理腳本、不接觸正式資料的內部工具 |
| 有條件使用 | 內部系統的非核心功能；必須補上測試、經 PR 與人工審查後才能合併 |
| 不得直接使用 | 登入與授權、金鑰與 Secret、個資與客戶資料、付款與財務、基礎架構與 CI/CD 設定；一律走完整 Lifecycle 與加強審查 |

## 原型轉為正式程式的條件

- ☐ 由負責的開發者完整讀過並能解釋程式邏輯
- ☐ 依團隊架構規範重構，不符 Pattern 的部分改寫
- ☐ 補齊單元測試與整合測試
- ☐ 通過 SAST、SCA 與 Secret Scanning
- ☐ 經 PR 與人工 Code Review 核准

Howard Liu Ph.D.

# 5　AI Coding Lifecycle 管理架構

受控的 AI Coding Lifecycle 讓 AI 在有限權限內產出，再由分支、PR、測試、安全檢查與受控發布決定變更能否推進。每個階段都要定義 AI 做什麼、人負責什麼，以及留下哪些紀錄。

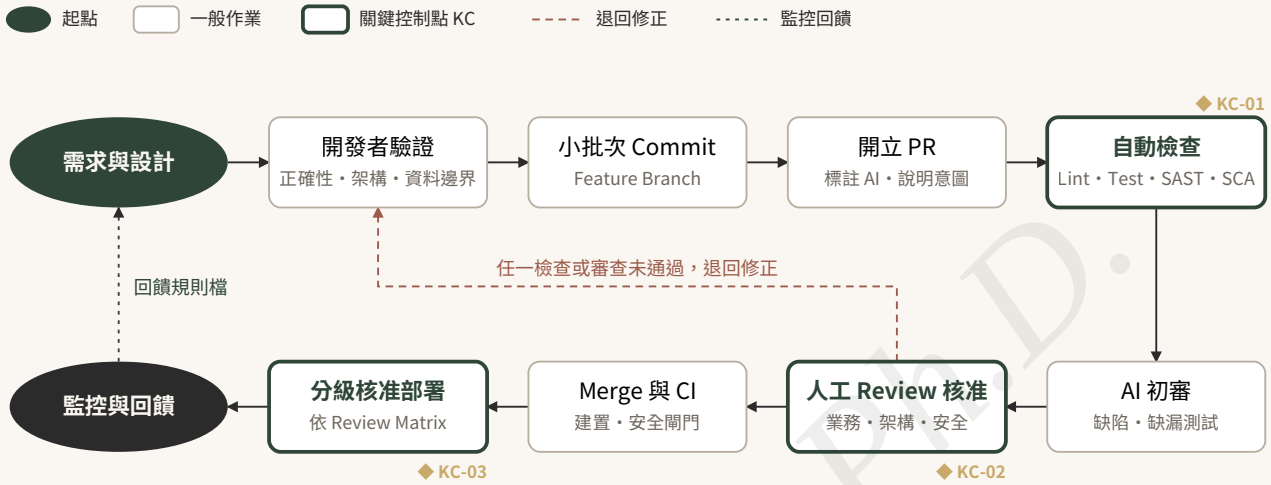



圖 5-1　AI Coding Lifecycle 標準流程：AI 只做初審，人工核准後才能合併上線

## 各階段責任分工

| 階段 | AI 的角色 | 人的責任 | 主要風險與控制 |
|---|---|---|---|
| **需求與設計** | 拆解 User Story、草擬驗收條件、比較技術方案 | 確認符合業務意圖與既有架構 | AI 幻覺或偏離架構 → 架構師確認 |
| **開發** | 補全、Chat、Agent、重構、產生文件 | 檢查正確性、架構與資料邊界 | 不理解就採用 → 開發者須能解釋每段程式 |
| **版控與 SCM** | 草擬 Commit 訊息與 PR 說明 | 小批次提交、標註 AI 輔助 | 大批次變更難審 → 限制 PR 大小 |
| **測試** | 產生測試案例與測試程式 | 確認涵蓋正常、邊界、異常與權限情境 | 測試只驗證 AI 自身假設 → 由人補關鍵案例 |
| **Code Review** | 初審：風格、明顯缺陷、缺漏測試、安全訊號 | 審查業務邏輯、架構、風險並做最終核准 | Review 疲勞 → AI 意見須引用具體 diff |
| **資安與品質閘門** | 協助解讀掃描結果、建議修正 | 決定阻擋或例外核准 | 依賴弱點與 Secret 外洩 → 掃描阻擋合併 |
| **部署與維運** | 協助分析日誌與事故 | 依風險分級核准部署、執行 Rollback | 高風險變更直接上線 → 分級核准與監控 |
| **回饋改善** | 彙整常見缺陷模式 | 更新 Prompt、規則檔與政策 | 同樣問題重複發生 → 檢討結果納入規則 |

Howard Liao Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。

# 6　Pattern & Framework 與 AI 程式品質審查

> Pattern 與 Framework 不是 AI 審查的硬性前提，但決定了審查的品質上限。規範文件化之後，AI 才有「對照基準」。

## 企業系統常見的 Pattern 與 Framework

| 類別 | 內容 |
|---|---|
| **應用框架** | 如 ASP.NET Core、Spring Boot、Django、NestJS；提供依賴注入、中介軟體、模型驗證與過濾器 |
| **分層架構** | Controller → Service → Repository；Controller 只接收參數與回傳結果，不寫 SQL 或業務計算 |
| **資料與交易** | Repository Pattern 與 Unit of Work；多表異動在同一交易中完成 |
| **統一回應格式** | Result／Envelope Pattern：成功旗標、代碼、訊息、資料、TraceId |
| **策略模式** | 把可擴充的業務檢核拆成獨立類別，新增規則時不修改核心程式 |
| **組態與韌性** | 強型別組態綁定、重試與斷路器、全域例外處理 |
| **測試框架** | 單元測試與 Mock 框架 |

## 有無 Pattern 時，AI 能審查什麼

| 審查面向 | 沒有統一 Pattern | 有 Pattern 且已文件化 |
|---|---|---|
| 語法錯誤與明顯 Bug | 可以 | 可以 |
| 常見資安弱點（SQL Injection、XSS） | 可以 | 可以，並指引改用框架標準寫法 |
| 例外與空值處理 | 只能給通用建議 | 依框架最佳實務精準修正 |
| 架構分層越界 | 無從判斷什麼算違規 | 可明確指出違規位置 |
| 交易一致性 | 不了解業務上的重要性 | 可指出違反 Unit of Work 規範 |
| 物件層級越權（BOLA） | 型別正確就視為合法 | 可指出缺少擁有者或權限驗證 |
| API 契約相容性 | 建議可能偏離既有標準 | 可指出未套用統一回應格式 |
| 自研框架的使用方式 | 完全不懂 | 提供文件與範例後可以審查 |

## 三個關鍵觀念

- **文件化才算數：**Pattern 若只存在資深工程師腦中，AI 看不到，必須寫成 AI 讀得到的規則檔與範例。
- **自研框架落差最大：**主流框架 AI 已有相當知識；內部框架與共用元件必須提供說明與範例，否則 AI 會用錯或建議繞過內部機制。

Howard Liao Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。

- **規範也防止過度設計：**沒有基準時，AI 容易把簡單邏輯包成 Factory 加 Strategy；明確規範讓 AI 在既定邊界內提出建議。

> Pattern 與 Framework 是施工藍圖，AI 是拿著藍圖驗收的監造。沒有藍圖，AI 只能檢查磚頭有沒有髒；有了藍圖，才能判斷整棟結構是否穩固。

# 7　審核程序與風險分級 Review Matrix

AI 不得自行合併主幹或核准上線；所有合併到主幹或與正式環境相關的變更，都需要具名人員核准。審核強度依變更風險分級如下：

| 等級 | 變更範例 | 自動檢查 | AI Review | 人工 Review | 額外核准 |
|---|---|---|---|---|---|
| 高 | 登入與授權、Secret、個資與客戶資料、付款、基礎架構、CI/CD 設定、對外 API | 全部必過，不得例外 | 必做，意見須逐項回覆 | 2 位 Reviewer，含 Code Owner | 資安代表或技術主管核准；部署需變更核准 |
| 中 | 核心業務邏輯、資料庫結構變更、第三方依賴升級 | 全部必過 | 必做 | 1 位 Reviewer，含 Code Owner | 技術主管知會 |
| 低 | UI 文字、內部工具、文件、測試程式 | 全部必過 | 建議 | 1 位 Reviewer | 無 |

## AI Review 的定位

- 導入初期作為**建議性檢查**，不作為阻擋條件；累積準確度資料後，再把高信心規則升級為阻擋條件。
- 負責風格、明顯缺陷、缺漏測試、安全訊號與違反規範的寫法。
- 意見必須引用具體 diff 位置並提供可執行的修正建議；泛泛的警告會造成 Review 疲勞。
- 人工 Reviewer 負責業務意圖、架構合理性、風險判斷與最終核准。

## 關鍵控制點

| 編號 | 控制點 | 類型 | 方式 | 頻率 | 負責人 |
|---|---|---|---|---|---|
| KC-01 | 合併前自動檢查與安全閘門 | 預防性 | 自動 | 每次 PR | DevOps 負責人 |
| KC-02 | 具名人工 Review 核准 | 預防性 | 人工 | 每次 PR | Code Owner |
| KC-03 | 依風險分級核准部署 | 預防性 | 混合 | 每次部署 | 技術主管 |
| KC-04 | AI 工具與資料邊界政策遵循檢查 | 偵測性 | 混合 | 每月 | 資安負責人 |
| KC-05 | 例外核准與事故事後檢討 | 矯正性 | 人工 | 事件發生時 | 導入負責人 |

## 必須保留的審核證據

PR 連結、AI 使用標註、測試結果、掃描報告、Reviewer 核准紀錄、部署紀錄，以及例外核准的理由與核准人。

Howard Liao Ph.D.

# 8　SCM 規範與稽核證據

AI 讓產出變快，SCM 規範的目的是讓每一筆變更仍然小到能審、清楚到能追。

| 項目 | 規範 |
| --- | --- |
| **分支策略** | 主幹受保護，禁止直接推送；一律經 Feature Branch 與 PR 合併 |
| **PR 大小** | 以小批次為原則，單一 PR 只處理一件事；過大的 PR 退回拆分 |
| **Commit 與 PR 描述** | 說明變更意圖與影響範圍，不只寫「AI 產生」 |
| **AI 使用標註** | PR 範本加入欄位：是否使用 AI、使用的工具、AI 參與的範圍 |
| **AI 工具權限** | Agent 只在隔離環境或 Feature Branch 執行，不持有主幹寫入與正式環境權限 |
| **依賴與版本** | 鎖定套件版本；新增依賴須經 SCA 掃描與授權檢查 |
| **Secret 管理** | 金鑰與密碼不得進入程式碼或 Prompt；啟用 Secret Scanning 並阻擋合併 |
| **資料邊界** | 正式資料、個資與機敏程式不得貼入未經核准的 AI 工具 |
| **規則檔版控** | AI 規則檔、Prompt 範本與 Review Rule 與程式碼一起版控，變更須經 PR |
| **稽核保存** | PR、測試、掃描、核准與部署紀錄依公司稽核政策保存 |

Howard Liao Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。

# 9　落地三步驟

讓 AI 成為有效的品質審查者，需要依序完成三件事：把規則寫下來、把機械式檢查交給工具、讓 AI 專注在語意判斷。

## 步驟一：規則文件化

在程式碼庫根目錄建立 AI 可讀的規則檔，例如 `CLAUDE.md`、`.cursorrules`、`.github/copilot-instructions.md`，或 AI Review 工具的規則設定。內容應包含架構原則、必用與禁用的寫法，以及正確與錯誤的範例。

```
# 程式碼規範與 AI 審查準則
1. 架構原則：遵守 Controller -> Service -> Repository 三層架構。
2. 禁止在 Controller 注入資料存取物件或直接執行 SQL。
3. 涉及業務檢核的流程，必須呼叫既定的檢核策略服務，不得自行實作。
4. 所有 API 回傳必須使用統一回應格式，並包含 TraceId。
5. 多表異動必須包在同一個交易內。
6. 存取資源前必須驗證目前使用者對該資源的權限。
```

## 步驟二：自動化靜態初篩

| 檢查項目 | 建議工具類型 | 為何不交給 AI |
| --- | --- | --- |
| 格式、命名、重複程式碼 | Linter、SonarQube | 規則明確，結果穩定且成本低 |
| 分層與依賴方向 | 架構測試（ArchUnit、NetArchTest、dependency-cruiser） | 可寫成測試，違規直接失敗 |
| 已知弱點與不安全寫法 | SAST | 規則庫成熟，結果可稽核 |
| 套件弱點與授權 | SCA、SBOM | 需比對弱點資料庫 |
| 金鑰外洩 | Secret Scanning | 必須完全阻擋 |

## 步驟三：AI 語意審查

- Pattern 用得是否合理，有沒有硬套或過度設計
- 業務邏輯是否放在正確的層級
- 交易邊界與一致性是否完整
- 物件層級權限是否確實驗證
- 例外與降級處理是否周全，例如外部服務中斷時的行為
- 測試是否真正涵蓋邊界與異常情境

AI 的錯誤建議要記錄下來，轉成規則檔中的「不要這樣建議」條目，持續降低雜訊。

Howard Liao Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。

# 10　專案組織與權責

導入負責人最適合由資深 Tech Lead 或架構師擔任，需同時具備 Code Review、Git 與 CI/CD 實務經驗。若找不到這樣的人選，可由資深開發主管搭配 DevOps 或資安代表共同負責。

## 角色職責

| 角色 | 在 AI Coding Lifecycle 中的責任 |
|---|---|
| **專案發起人（主管）** | 核准 AI 工具與授權、追蹤試行指標、決定是否擴大導入 |
| **導入負責人（Tech Lead／架構師）** | 制定架構規範與規則檔、定義 Review Matrix、擔任高風險變更的 Code Owner |
| **開發者** | 理解並驗證 AI 產出、小批次提交、標註 AI 使用、對提交的程式負責 |
| **Reviewer／Code Owner** | 審查業務邏輯與架構、回應 AI 審查意見、做出核准決定 |
| **DevOps／CI-CD 負責人** | 建置自動檢查與安全閘門、環境隔離、部署與 Rollback 機制 |
| **資安負責人** | 定義資料邊界與 AI 使用政策、管理掃描工具、核准高風險例外 |
| **QA／測試負責人** | 評估 AI 生成測試的有效性與覆蓋率、補充關鍵測試案例 |

## RACI 矩陣

R＝執行　A＝當責　C＝諮詢　I＝知會

| 工作項目 | 主管 | 導入負責人 | 開發者 | Reviewer | DevOps | 資安 | QA |
|---|---|---|---|---|---|---|---|
| AI 使用政策 | A | R | I | I | C | R | I |
| 規範文件化與規則檔 | I | A/R | C | C | I | C | C |
| 自動檢查與安全閘門 | I | A | I | I | R | R | C |
| PR 與 Code Review | I | A | R | R | I | C | C |
| 分級核准部署 | I | A | I | C | R | C | C |
| 試行指標追蹤與評估 | A | R | I | I | R | C | C |

Howard Liao Ph.D.

# 11　專案時程與里程碑

建議以 10 週完成導入與試行：前 5 週建立規範與工具，第 6 至 9 週試行兩個 Sprint，第 10 週評估並決定是否擴大。實際起始日依專案核准日調整。

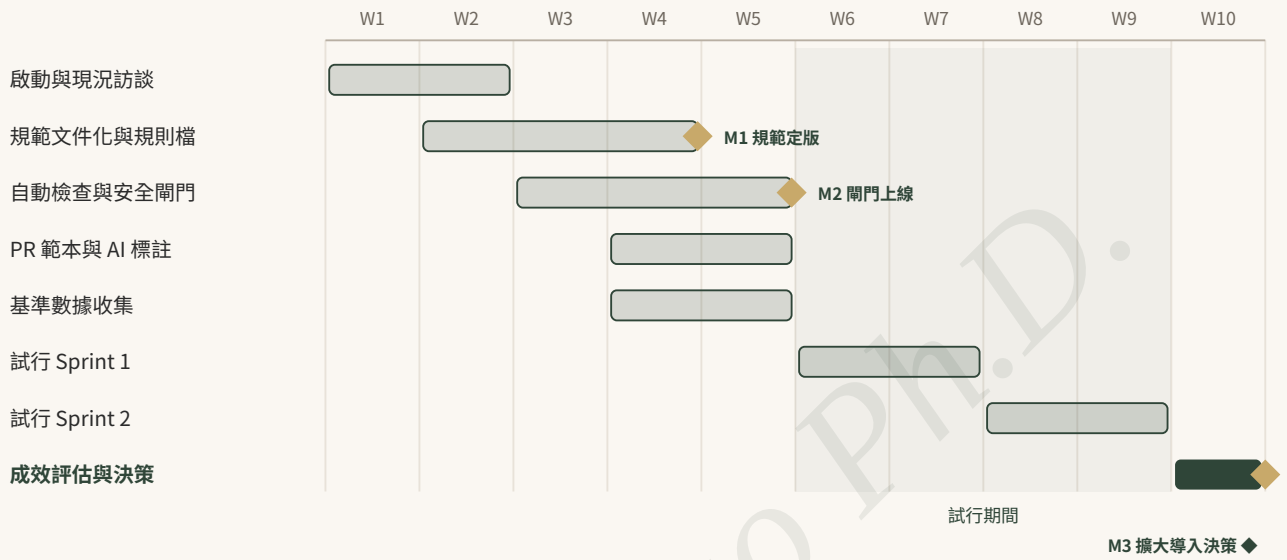



圖 11-1　專案時程（建議）：前 5 週建立規範與工具，後 5 週試行與評估

## 里程碑與驗收條件

| 編號 | 里程碑 | 時點 | 驗收條件 |
|---|---|---|---|
| M1 | 規範定版 | 第 4 週末 | AI 使用政策、Lifecycle、Review Matrix、SCM 規範與規則檔經核定 |
| M2 | 閘門上線 | 第 5 週末 | 試行系統的保護分支、PR 範本、自動檢查與安全閘門全部啟用 |
| M3 | 擴大導入決策 | 第 10 週末 | 完成試行評估報告，由專案發起人決定擴大、調整或暫停 |

Howard Liao Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。

# 12　成效衡量指標

試行前先收集一至兩個 Sprint 的基準值，試行後比較差異。速度提升但失敗率或返工率同時上升，代表品質成本被轉嫁到後段，應先修正流程再擴大導入。

| 指標 | 意義 | 期望方向 |
| --- | --- | --- |
| Lead Time for Changes | 從 Commit 到上線的時間 | 縮短 |
| Change Failure Rate | 上線後造成事故或需回滾的比例 | 不上升 |
| Code Churn Rate | 程式提交後短期內被改寫的比例 | 不上升 |
| Review Time | PR 從開立到核准的時間 | 不因 AI 產出增加而拉長 |
| PR 大小 | 單一 PR 的變更行數 | 維持小批次 |
| AI Review 採納率 | AI 意見被採納的比例 | 逐步提升，代表雜訊下降 |
| 資安事件與掃描阻擋數 | 進入主幹前被攔截與漏網的問題 | 漏網為零 |

Howard Liao Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。

# 13　專案風險與因應

| 風險 | 等級 | 可能影響 | 因應措施 |
|---|---|---|---|
| 機敏資料或 Secret 進入 AI 工具 | 高 | 資料外洩、法遵責任 | 限定核准工具、資料邊界政策、Secret Scanning、定期遵循檢查 |
| 人工 Review 成為瓶頸 | 中 | 交付延遲、審查流於形式 | 限制 PR 大小、AI 初審分流、依風險分級審查 |
| AI 審查雜訊過多 | 中 | 工程師忽略 AI 意見 | 先作建議性檢查、要求引用 diff、錯誤建議轉成否定規則 |
| 規範文件化進度落後 | 中 | AI 審查缺乏基準 | 先完成最關鍵 5 至 6 條規則，其餘於試行中補齊 |
| 工程師接受度不足 | 低 | 流程未被落實 | 導入負責人帶頭示範、每週檢討分享案例、以數據說明成效 |
| 關鍵人員依賴 | 中 | 負責人離開後規範失傳 | 規則檔與流程版控、指定代理人、定期交接演練 |

Howard Liao Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。

# 14　專案交付成果

| # | 交付成果 | 內容 | 完成時點 |
|---|---|---|---|
| 1 | AI Coding 使用政策 | 核准工具、Vibe Coding 邊界、資料與 Secret 邊界 | 第 4 週 |
| 2 | AI Coding Lifecycle Map | 各階段 AI 使用方式、輸入、輸出、負責人與風險 | 第 4 週 |
| 3 | Review Matrix | 依風險等級定義自動檢查、AI Review、人工 Review 與額外核准 | 第 4 週 |
| 4 | SCM 規範 | 分支策略、Commit／PR 標準、AI 標註、版本鎖定、稽核證據 | 第 4 週 |
| 5 | AI 規則檔與 PR 範本 | 架構規範、禁用寫法、範例程式；PR 的 AI 使用欄位 | 第 5 週 |
| 6 | 自動檢查與安全閘門設定 | Linter、架構測試、SAST、SCA、Secret Scanning | 第 5 週 |
| 7 | 試行評估報告 | 指標前後比較、問題清單、擴大導入建議 | 第 10 週 |

Howard Liao Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。

# 15　結論與下一步

Vibe Coding 帶來速度，AI Coding Lifecycle 管理帶來可控性，兩者並不衝突。關鍵在於劃清邊界：原型可以快，正式系統必須經過規範、測試、審查與核准。

AI 審查的價值，取決於團隊是否先把 Pattern 與 Framework 寫成 AI 讀得到的規則。建議依下列順序推進：

1. 核定本專案，指定導入負責人與專案小組成員。
2. 選定一個低風險試行系統，收集導入前的基準數據。
3. 完成規則文件化與 PR 範本調整（M1）。
4. 啟用自動檢查與安全閘門（M2）。
5. 試行兩個 Sprint 後提出評估報告，決定是否擴大導入（M3）。

Howard Liao Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。

# 附錄　訪談與檢核題綱、參考資料

以下題目可用於訪談導入負責人，也可用於檢核內部團隊或外部協力廠商交付的程式品質。

### 現況與治理

1. 目前由誰負責 AI 工具的選擇、授權、政策與稽核？
2. 團隊是否有使用 Vibe Coding？用在哪些範圍？原型轉正式系統時有什麼把關？
3. 是否有標註、追蹤或統計 AI 輔助程式碼的方式？

### Lifecycle 各階段

1. 需求與設計階段如何使用 AI？如何避免不符架構的建議？
2. 開發時使用哪些 AI 模式？Prompt 是否版控？
3. AI 產生的測試如何驗證？是否涵蓋正常、邊界、異常與權限情境？
4. AI 相關變更上線前是否需要額外核准？Rollback 與事故檢討如何進行？

### Pattern、Framework 與 AI 審查

1. 是否有統一的架構風格與常用 Design Pattern？是否有文件與範例程式？
2. 是否有自研框架或共用元件？AI 工具讀得到它們的說明嗎？
3. Coding Guideline 是否已轉成 AI 可讀的規則檔？由誰維護、多久更新？
4. 哪些規則已由 Linter、編譯器或架構測試強制？哪些只能靠人工或 AI 判斷？
5. AI 是否曾給出違反團隊架構的建議？是否有機制把它轉成否定規則？

### 審核與交接

1. AI 能否自行合併？哪些變更需要提高審核強度？
2. 審核證據保存了哪些項目？能否在稽核時完整提出？
3. 程式是否具備足夠的規範一致性，讓接手團隊能在 AI 輔助下快速理解並維護？

## 參考資料


- DORA, State of AI-assisted Software Development 2025 — dora.dev/dora-report-2025
- Scrum.org, DORA Report 2025 Summary — scrum.org
- Resolute Software, AI Governance SDLC Controls — resolutesoftware.com
- Northflank, AI-powered Secure SDLC — northflank.com
- AI SDLC — aisdlc.io
- Collin Wilkins, AI Code Review Best Practices — collinwilkins.com


— 以下空白 —

Howard Lee Ph.D.

本文件為內部管制文件，未經核准不得複製或對外提供。